

LAPORAN PRAKTIKUM
STRUKTUR DATA
IMPLEMENTASI BTREE DAN GRAPH



Disusun Oleh:

Gina Ramadhani

Nim: 2511533014

Dosen Pengampu: DR. Wahyudi, S.T, M.T

Asisten Praktikum: Sherly Sukmadira

DAPERTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
TAHUN 2026

KATA PENGANTAR

Puji syukur ke hadirat Allah SWT karena atas rahmat dan karunia-Nya laporan praktikum Struktur Data mengenai Binary Tree (BTree) dan Graph Traversal ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai salah satu tugas praktikum untuk memahami konsep struktur data non-linear serta implementasi algoritma penelusuran pada pohon dan graf menggunakan bahasa pemrograman Java.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi perbaikan di masa mendatang. Semoga laporan ini dapat memberikan manfaat serta menambah pemahaman mengenai Binary Tree dan Graph Traversal.

Padang, 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Tujuan Praktikum	1
1.3 Manfaat Praktikum	1
BAB II PEMBAHASAN.....	3
2.1 Dasar Teori	3
2.2 Praktikum Class “Node_2511533014”	4
2.3 Praktikum Class “BTree_2511533014”	7
2.4 Praktikum Class “BTreeDriver_2511533014”	11
2.5 Praktikum Class “GraphTraversal_2511533014”	16
2.6 Laporan Tugas Praktikum	21
BAB III PENUTUP.....	29
3.1 Kesimpulan	29
DAFTAR PUSTAKA.....	30

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan salah satu konsep penting dalam ilmu komputer yang digunakan untuk mengorganisasi dan mengelola data agar dapat diakses serta diproses secara efisien. Selain struktur data linear seperti array dan linked list, terdapat struktur data non-linear yang mampu merepresentasikan hubungan data yang lebih kompleks, di antaranya adalah Binary Tree dan Graph.

Binary Tree digunakan untuk menyimpan data dalam bentuk hierarki yang terdiri dari root, parent, dan child node. Sementara itu, Graph digunakan untuk merepresentasikan hubungan antar objek yang saling terhubung melalui simpul (vertex) dan sisi (edge). Untuk melakukan penelusuran pada graph, digunakan algoritma Breadth First Search (BFS) dan Depth First Search (DFS). Oleh karena itu, praktikum ini dilakukan untuk memahami konsep serta implementasi Binary Tree dan Graph Traversal menggunakan bahasa pemrograman Java.

1.2 Tujuan Praktikum

1. Mempelajari cara membangun dan mengelola struktur Binary Tree menggunakan Java.
2. Memahami proses traversal pada Binary Tree menggunakan metode Preorder, Inorder, dan Postorder.
3. Mengimplementasikan algoritma Breadth First Search (BFS) dan Depth First Search (DFS) pada Graph.
4. Menganalisis hasil penelusuran data pada Binary Tree dan Graph.

1.3 Manfaat Praktikum

1. Menambah pemahaman mengenai struktur data non-linear.
2. Melatih kemampuan pemrograman Java dalam implementasi Binary Tree dan Graph.
3. Memahami cara kerja algoritma traversal pada Binary Tree.

4. Menjadi dasar dalam pengembangan aplikasi yang memanfaatkan struktur pohon dan graf.

BAB II PEMBAHASAN

2.1 Dasar Teori

Node (simpul) merupakan elemen dasar dalam struktur data yang digunakan untuk menyimpan data dan membentuk hubungan dengan node lainnya. Pada struktur data non-linear seperti tree dan graph, node berperan sebagai komponen utama yang menyimpan informasi serta menghubungkan bagian-bagian struktur data.

B-Tree (Balanced Tree) adalah struktur data non-linear berbentuk pohon yang dirancang untuk menyimpan dan mengelola data dalam jumlah besar secara efisien. Setiap node pada B-Tree dapat menyimpan beberapa key (kunci) dan memiliki lebih dari dua child (anak). B-Tree menjaga keseimbangan pohon sehingga proses pencarian, penyisipan, dan penghapusan data dapat dilakukan dengan cepat. Karena kemampuannya tersebut, B-Tree banyak digunakan pada sistem basis data dan sistem file.

Selain *tree*, struktur data *non-linear* lainnya adalah *graph*. *Graph* merupakan kumpulan *node (vertex)* yang saling terhubung melalui *edge (sisi)*. Untuk mengakses atau mencari data pada graph digunakan proses Graph Traversal, yaitu teknik menelusuri seluruh node dalam graph secara sistematis. Dua metode traversal yang umum digunakan adalah *Depth First Search (DFS)* dan *Breadth First Search (BFS)*. DFS menelusuri graph hingga kedalaman maksimum sebelum kembali ke node sebelumnya, sedangkan BFS menelusuri *graph* berdasarkan tingkat kedekatan node dari titik awal menggunakan antrian (*queue*).

Secara umum, node merupakan komponen dasar yang membentuk B-Tree dan Graph. B-Tree digunakan untuk mengelola data secara terstruktur dan seimbang, sedangkan Graph Traversal digunakan untuk menelusuri hubungan antar node dalam suatu graph melalui algoritma DFS dan BFS.

2.2 Praktikum Class “Node_2511533014”

```
1 package pekan9_2511533014;
2
3 public class Node_2511533014 {
4     int data_3014;
5     Node_2511533014 left_3014;
6     Node_2511533014 right_3014;
7     public Node_2511533014 (int data_3014) {
8         this.data_3014 = data_3014;
9         left_3014 = null;
10        right_3014 = null;
11    }
12    public void setLeft_3014(Node_2511533014 node_3014) {
13        if (left_3014 == null)
14            left_3014 = node_3014;
15    }
16    public void setRight_3014(Node_2511533014 node_3014) {
17        if (right_3014 == null)
18            right_3014 = node_3014;
19    }
20    public Node_2511533014 getLeft_3014() {
21        return left_3014;
22    }
23    public Node_2511533014 getRight_3014 () {
24        return right_3014;
25    }
26
27    public int getData_3014() {
28        return data_3014;
29    }
30    public void setData_3014 () {
31        this.data_3014 = data_3014;
32    }
33
34    void printPreorder_3014(Node_2511533014 node_3014) {
35        if (node_3014 == null)
36            return;
37        System.out.print(node_3014.data_3014 + " ");
38        printPreorder_3014 (node_3014.left_3014);
39        printPreorder_3014 (node_3014.right_3014);
40    }
41    void printPostorder_3014(Node_2511533014 node_3014) {
42        if (node_3014 == null)
43            return;
44        printPostorder_3014(node_3014.left_3014);
45        printPostorder_3014(node_3014.right_3014);
46        System.out.print(node_3014.data_3014 + " ");
47    }
48    void printInorder_3014(Node_2511533014 node_3014) {
49        if (node_3014 == null)
50            return;
51        printInorder_3014(node_3014.left_3014);
52        System.out.print(node_3014.data_3014 + " ");
53        printInorder_3014(node_3014.right_3014);
54    }
55    public String print_3014() {
56        return this.print_3014("",true,"");
57    }
58    public String print_3014(String prefix, boolean isTail, String sb) {
59        if (right_3014 != null) {
60            right_3014.print_3014(prefix + (isTail ? "| " : " "), false, sb);
61        }
62        System.out.println( prefix+(isTail ? "\\--" : "/--")+data_3014);
63        if (left_3014 != null) {
64            left_3014.print_3014(prefix+(isTail ? " " : "| "), true, sb);
65        }
66        return sb;
67    }
68 }
```

Gambar 2.1 Kode program praktikum class “Node”

Program `Node_2511533014` merupakan kelas yang digunakan untuk merepresentasikan node pada Binary Tree. Setiap node menyimpan sebuah data bertipe integer (`data_3014`) serta memiliki referensi ke anak kiri (`left_3014`) dan anak kanan (`right_3014`). Kelas ini menyediakan method untuk menambah dan mengakses child node, menampilkan isi pohon menggunakan traversal Preorder, Inorder, dan Postorder, serta mencetak struktur pohon secara visual.

Berikut penjelasan langkah-langkah kerjanya:

1. `int data_3014;`
→ Menyimpan nilai data yang terdapat pada node.
2. `Node_2511533014 left_3014;`
→ Menyimpan referensi atau alamat child (anak) kiri.
3. `Node_2511533014 right_3014;`
→ Menyimpan referensi atau alamat child (anak) kanan.
4. `public Node_2511533014(int data_3014)`
→ Konstruktor untuk membuat node baru dengan nilai tertentu.
5. `this.data_3014 = data_3014;`
→ Mengisi data node dengan nilai yang diberikan saat objek dibuat.
6. `left_3014 = null;`
→ Menginisialisasi child kiri dengan nilai kosong (belum ada node).
7. `right_3014 = null;`
→ Menginisialisasi child kanan dengan nilai kosong (belum ada node).
8. `public void setLeft_3014(Node_2511533014 node_3014)`
→ Method untuk menambahkan child kiri pada node.
9. `public void setRight_3014(Node_2511533014 node_3014)`
→ Method untuk menambahkan child kanan pada node.

10. `public Node_2511533014 getLeft_3014()`
→ Mengambil atau mengembalikan child kiri dari node.
11. `public Node_2511533014 getRight_3014()`
→ Mengambil atau mengembalikan child kanan dari node.
12. `public int getData_3014()`
→ Mengambil nilai data yang tersimpan pada node.
13. `void printPreorder_3014(Node_2511533014 node_3014)`
→ Menampilkan isi pohon dengan traversal Preorder (Root → Left → Right).
14. `System.out.print(node_3014.data_3014 + " ");`
→ Mencetak data node yang sedang dikunjungi.
15. `printPreorder_3014(node_3014.left_3014);`
→ Menelusuri subtree kiri secara rekursif.
16. `printPreorder_3014(node_3014.right_3014);`
→ Menelusuri subtree kanan secara rekursif.
17. `void printPostorder_3014(Node_2511533014 node_3014)`
→ Menampilkan isi pohon dengan traversal Postorder (Left → Right → Root).
18. `void printInorder_3014(Node_2511533014 node_3014)`
→ Menampilkan isi pohon dengan traversal Inorder (Left → Root → Right).
19. `public String print_3014()`
→ Method untuk mencetak struktur Binary Tree secara visual.
20. `System.out.println(prefix + (isTail ? "\\--" : "/--") + data_3014);`
→ Menampilkan node beserta cabang-cabangnya dalam bentuk diagram pohon.

2.3 Praktikum Class “BTree_2511533014”

```
1 package pekan9_2511533014;
2
3 public class BTree_2511533014 {
4     private Node_2511533014 root_3014;
5     private Node_2511533014 currentNode_3014;
6     public BTree_2511533014() {
7         root_3014 = null;
8     }
9     public boolean search_3014(int data_3014) {
10         return search_3014(root_3014, data_3014);
11     }
12     private boolean search_3014(Node_2511533014 node_3014, int data_3014) {
13         if (node_3014.getData_3014() == data_3014)
14             return true;
15         if (node_3014.getLeft_3014() != null)
16             if (search_3014(node_3014.getLeft_3014(), data_3014))
17                 return true;
18         if (node_3014.getRight_3014() != null)
19             if (search_3014(node_3014.getRight_3014(), data_3014))
20                 return true;
21         return false;
22     }
23     public void printInorder_3014() {
24         root_3014.printInorder_3014(root_3014);
25     }
26     public void printPreorder_3014() {
27         root_3014.printPreorder_3014(root_3014);
28     }
29     public void printPostorder_3014() {
30         root_3014.printPostorder_3014(root_3014);
31     }
32     public Node_2511533014 getRoot_3014() {
33         return root_3014;
34     }
35     public boolean isEmpty_3014() {
36         return root_3014 == null;
37     }
38     public int countNodes_3014() {
39         return countNodes_3014(root_3014);
40     }
41     private int countNodes_3014(Node_2511533014 node_3014) {
42         int count_3014 = 1;
43         if (node_3014 == null) {
44             return 0;
45         } else {
46             count_3014 += countNodes_3014(node_3014.getLeft_3014());
47             count_3014 += countNodes_3014(node_3014.getRight_3014());
48             return count_3014;
49         }
50     }
51     public void print_3014() {
52         root_3014.print_3014();
53     }
54     public Node_2511533014 getCurrent_3014() {
55         return currentNode_3014;
56     }
57     public void setCurrent_3014(Node_2511533014 node_3014) {
58         this.currentNode_3014 = node_3014;
59     }
60     public void setRoot_3014(Node_2511533014 root_3014) {
61         this.root_3014 = root_3014;
62     }
63 }
```

Gambar 2.2 Kode Program praktikum class “BTree”

Program BTree_2511533014 merupakan kelas yang digunakan untuk mengelola struktur data *Binary Tree*. Kelas ini memiliki root sebagai akar pohon dan menyediakan berbagai method untuk melakukan pencarian data (*search*), menghitung jumlah node, menampilkan isi pohon dengan traversal (*Inorder*, *Preorder*, *Postorder*), serta mengatur node root dan node yang sedang aktif (*current node*).

Berikut penjelasan langkah-langkah kerjanya:

1. private Node_2511533014 root_3014;
→ Menyimpan node akar (root) dari Binary Tree.
2. private Node_2511533014 currentNode_3014;
→ Menyimpan node yang sedang aktif atau sedang diproses.
3. public BTree_2511533014()
→ Konstruktor untuk membuat objek Binary Tree.
4. root_3014 = null;
→ Menginisialisasi tree dalam keadaan kosong.
5. public boolean search_3014(int data_3014)
→ Method untuk mencari data pada Binary Tree.
6. return search_3014(root_3014, data_3014);
→ Memulai pencarian dari node root.
7. private boolean search_3014(Node_2511533014 node_3014, int data_3014)
→ Method rekursif untuk mencari data pada seluruh node.
8. if (node_3014.getData_3014() == data_3014)
→ Memeriksa apakah data yang dicari ditemukan.
9. return true;
→ Mengembalikan nilai benar jika data ditemukan.

10. `if (node_3014.getLeft_3014() != null)`
→ Memeriksa apakah terdapat child kiri.
11. `search_3014(node_3014.getLeft_3014(), data_3014)`
→ Melakukan pencarian pada subtree kiri.
12. `if (node_3014.getRight_3014() != null)`
→ Memeriksa apakah terdapat child kanan.
13. `search_3014(node_3014.getRight_3014(), data_3014)`
→ Melakukan pencarian pada subtree kanan.
14. `return false;`
→ Mengembalikan nilai salah jika data tidak ditemukan.
15. `public void printInorder_3014()`
→ Menampilkan data pohon dengan traversal Inorder.
16. `root_3014.printInorder_3014(root_3014);`
→ Memulai traversal Inorder dari root.
17. `public void printPreorder_3014()`
→ Menampilkan data pohon dengan traversal Preorder.
18. `root_3014.printPreorder_3014(root_3014);`
→ Memulai traversal Preorder dari root.
19. `public void printPostorder_3014()`
→ Menampilkan data pohon dengan traversal Postorder.
20. `root_3014.printPostorder_3014(root_3014);`
→ Memulai traversal Postorder dari root.
21. `public Node_2511533014 getRoot_3014()`
→ Mengambil node root dari Binary Tree.

22. `public boolean isEmpty_3014()`
→ Memeriksa apakah Binary Tree kosong.
23. `return root_3014 == null;`
→ Mengembalikan nilai true jika root kosong.
24. `public int countNodes_3014()`
→ Menghitung jumlah seluruh node pada Binary Tree.
25. `return countNodes_3014(root_3014);`
→ Memulai proses perhitungan dari root.
26. `private int countNodes_3014(Node_2511533014 node_3014)`
→ Method rekursif untuk menghitung jumlah node.
27. `int count_3014 = 1;`
→ Menginisialisasi penghitung untuk node saat ini.
28. `if (node_3014 == null)`
→ Memeriksa apakah node kosong.
29. `countNodes_3014(node_3014.getLeft_3014());`
→ Menghitung jumlah node pada subtree kiri.
30. `countNodes_3014(node_3014.getRight_3014());`
→ Menghitung jumlah node pada subtree kanan.
31. `public void print_3014()`
→ Menampilkan struktur Binary Tree secara visual.
32. `root_3014.print_3014();`
→ Memanggil method cetak dari node root.
33. `public Node_2511533014 getCurrent_3014()`
→ Mengambil node yang sedang aktif.

34. public void setCurrent_3014(Node_2511533014 node_3014)

→ Mengubah node yang sedang aktif.

35. this.currentNode_3014 = node_3014;

→ Menyimpan node ke variabel currentNode_3014.

36. public void setRoot_3014(Node_2511533014 root_3014)

→ Mengubah atau menetapkan node root.

37. this.root_3014 = root_3014;

→ Menyimpan node sebagai akar Binary Tree.

2.4 Praktikum Class “BTreeDriver_2511533014”

```
1 package pekan9_2511533014;
2
3 public class BTreeDriver_2511533014 {
4     public static void main(String[] args_3014) {
5         // Membuat Pohon
6         BTree_2511533014 tree_3014 = new BTree_2511533014();
7         System.out.print("Jumlah Simpul awal pohon: ");
8         System.out.println(tree_3014.countNodes_3014());
9         // menambahkan simpul data 1
10        Node_2511533014 root_3014 = new Node_2511533014(1);
11        // menjadikan simpul 1 sebagai root
12        tree_3014.setRoot_3014(root_3014);
13        System.out.println("Jumlah simpul jika hanya ada root");
14        System.out.println(tree_3014.countNodes_3014());
15        Node_2511533014 node2_3014 = new Node_2511533014(2);
16        Node_2511533014 node3_3014 = new Node_2511533014(3);
17        Node_2511533014 node4_3014 = new Node_2511533014(4);
18        Node_2511533014 node5_3014 = new Node_2511533014(5);
19        Node_2511533014 node6_3014 = new Node_2511533014(6);
20        Node_2511533014 node7_3014 = new Node_2511533014(7);
21        Node_2511533014 node8_3014 = new Node_2511533014(8);
22        Node_2511533014 node9_3014 = new Node_2511533014(9);
23        root_3014.setLeft_3014(node2_3014);
24        node2_3014.setLeft_3014(node4_3014);
25        node2_3014.setRight_3014(node5_3014);
26        node4_3014.setRight_3014(node8_3014);
27        root_3014.setRight_3014(node3_3014);
28        node3_3014.setLeft_3014(node6_3014);
29        node3_3014.setRight_3014(node7_3014);
30        node6_3014.setLeft_3014(node9_3014);
```

```

31         //Set root
32         tree_3014.setCurrent_3014(tree_3014.getRoot_3014());
33         System.out.println("Menampilkan simpul terakhir: ");
34         System.out.println(tree_3014.getCurrent_3014().getData_3014());
35         System.out.println("Jumlah simpul; setelah simpul 7 ditambahkan");
36         System.out.println(tree_3014.countNodes_3014());
37         System.out.println("InOrder: ");
38         tree_3014.printInorder_3014();
39         System.out.println("\nPreorder: ");
40         tree_3014.printPreorder_3014();
41         System.out.println("\nPostorder : ");
42         tree_3014.printPostorder_3014();
43         System.out.println("\nMenampilkan simpul dalam bentuk pohon");
44         tree_3014.print_3014();
45     }
46 }

```

Gambar 2.3 Kode Program praktikum class “BTreeDriver”

Program BTreeDriver_2511533014 merupakan kelas utama (*driver class*) yang digunakan untuk menjalankan dan menguji operasi pada struktur data Binary Tree. Program ini membuat node-node baru, menyusun hubungan antar node menjadi sebuah pohon biner, menghitung jumlah node, menampilkan data menggunakan traversal Inorder, Preorder, dan Postorder, serta mencetak struktur pohon secara visual.

Berikut penjelasan langkah-langkah kerjanya:

1. `public static void main(String[] args_3014)`
→ Method utama yang dijalankan pertama kali saat program dieksekusi.
2. `BTree_2511533014 tree_3014 = new BTree_2511533014();`
→ Membuat objek Binary Tree baru.
3. `tree_3014.countNodes_3014();`
→ Menghitung jumlah node yang ada dalam tree.
4. `Node_2511533014 root_3014 = new Node_2511533014(1);`
→ Membuat node dengan data 1 yang akan dijadikan root.
5. `tree_3014.setRoot_3014(root_3014);`
→ Menetapkan node 1 sebagai root Binary Tree.
6. `Node_2511533014 node2_3014 = new Node_2511533014(2);`

- Membuat node dengan data 2.
- 7. Node_2511533014 node3_3014 = new Node_2511533014(3);
 - Membuat node dengan data 3.
- 8. Node_2511533014 node4_3014 = new Node_2511533014(4);
 - Membuat node dengan data 4.
- 9. Node_2511533014 node5_3014 = new Node_2511533014(5);
 - Membuat node dengan data 5.
- 10. Node_2511533014 node6_3014 = new Node_2511533014(6);
 - Membuat node dengan data 6.
- 11. Node_2511533014 node7_3014 = new Node_2511533014(7);
 - Membuat node dengan data 7.
- 12. Node_2511533014 node8_3014 = new Node_2511533014(8);
 - Membuat node dengan data 8.
- 13. Node_2511533014 node9_3014 = new Node_2511533014(9);
 - Membuat node dengan data 9.
- 14. root_3014.setLeft_3014(node2_3014);
 - Menjadikan node 2 sebagai child kiri root.
- 15. root_3014.setRight_3014(node3_3014);
 - Menjadikan node 3 sebagai child kanan root.
- 16. node2_3014.setLeft_3014(node4_3014);
 - Menjadikan node 4 sebagai child kiri node 2.
- 17. node2_3014.setRight_3014(node5_3014);
 - Menjadikan node 5 sebagai child kanan node 2.
- 18. node4_3014.setRight_3014(node8_3014);

- Menjadikan node 8 sebagai child kanan node 4.
- 19. `node3_3014.setLeft_3014(node6_3014);`
 - Menjadikan node 6 sebagai child kiri node 3.
- 20. `node3_3014.setRight_3014(node7_3014);`
 - Menjadikan node 7 sebagai child kanan node 3.
- 21. `node6_3014.setLeft_3014(node9_3014);`
 - Menjadikan node 9 sebagai child kiri node 6.
- 22. `tree_3014.setCurrent_3014(tree_3014.getRoot_3014());`
 - Mengatur node root sebagai current node.
- 23. `tree_3014.getCurrent_3014().getData_3014();`
 - Mengambil data dari current node.
- 24. `tree_3014.printInorder_3014();`
 - Menampilkan isi tree dengan traversal Inorder (Left–Root–Right).
- 25. `tree_3014.printPreorder_3014();`
 - Menampilkan isi tree dengan traversal Preorder (Root–Left–Right).
- 26. `tree_3014.printPostorder_3014();`
 - Menampilkan isi tree dengan traversal Postorder (Left–Right–Root).
- 27. `tree_3014.print_3014();`
 - Menampilkan struktur Binary Tree dalam bentuk diagram pohon.

Dari Langkah-langkah diatas kita akan mendapatkan output seperti gambar 2.4 dibawah ini.

```

<terminated> BTreeDriver_2511533014 [Java Application] C:\U
Jumlah Simpul awal pohon: 0
Jumlah simpul jika hanya ada root
1
Menampilkan simpul terakhir:
1
Jumlah simpul; setelah simpul 7 ditambahkan
9
InOrder:
4 8 2 5 1 9 6 3 7
Preorder:
1 2 4 8 5 3 6 9 7
Postorder :
8 4 5 2 9 6 7 3 1
Dmenampilkan simpul dalam bentuk pohon
|   /--7
|  /--3
|  |  \--6
|  |  \--9
|--1
|  /--5
|--2
|  /--8
|--4

```

**Gambar 2.4 Output kode program praktikum
class “BteeDriver”**

2.1.1 Analisis

Berdasarkan output yang dihasilkan, program berhasil membentuk *Binary Tree* dengan 9 simpul. Awalnya jumlah simpul bernilai 0 karena pohon masih kosong. Setelah *node* 1 ditetapkan sebagai *root*, jumlah simpul menjadi 1. Selanjutnya ditambahkan node 2 hingga 9 sehingga total simpul menjadi 9. Hasil *traversal Inorder* menghasilkan urutan 4 8 2 5 1 9 6 3 7, yang menunjukkan proses penelusuran dari *subtree* kiri, root, lalu subtree kanan. Traversal Preorder menghasilkan urutan 1 2 4 8 5 3 6 9 7, yang dimulai dari root kemudian menelusuri subtree kiri dan kanan. Sementara itu, traversal Postorder menghasilkan urutan 8 4 5 2 9 6 7 3 1, yang menunjukkan bahwa seluruh child dikunjungi terlebih dahulu sebelum root. Tampilan pohon pada bagian akhir juga sesuai dengan struktur Binary Tree yang telah dibentuk, dengan node 1 sebagai root, node 2 dan 3 sebagai child utama, serta node 4, 5, 6, 7, 8, dan 9 sebagai cabang-cabang turunannya. Dengan demikian, seluruh operasi pembentukan pohon, perhitungan jumlah simpul, dan traversal telah berjalan dengan benar.

2.5 Praktikum Class “GraphTraversal_2511533014”

```
1 package pekan9_2511533014;
2 import java.util.*;
3 public class GraphTraversal_2511533014 {
4     private Map<String, List<String>> graph_3014 = new HashMap<>();
5
6     // Menambahkan edge (graf tak berarah)
7     public void addEdge_3014(String node1_3014, String node2_3014) {
8         graph_3014.putIfAbsent(node1_3014, new ArrayList<>());
9         graph_3014.putIfAbsent(node2_3014, new ArrayList<>());
10        graph_3014.get(node1_3014).add(node2_3014);
11        graph_3014.get(node2_3014).add(node1_3014);
12    }
13
14    // Menampilkan graf awal
15    public void printGraph_3014() {
16        System.out.println("Graf Awal (Adjacency List):");
17        for (String node_3014 : graph_3014.keySet()) {
18            System.out.print(node_3014 + " -> ");
19            List<String> neighbors_3014 = graph_3014.get(node_3014);
20            System.out.println(String.join(", ", neighbors_3014));
21        }
22        System.out.println();
23    }
24
25    // DFS rekursif
26    public void dfs_3014(String start_3014) {
27        Set<String> visited_3014 = new HashSet<>();
28        System.out.println("Penelusuran DFS:");
29        dfsHelper_3014(start_3014, visited_3014);
30        System.out.println();
31    }
32
33    private void dfsHelper_3014(String current_3014, Set<String> visited_3014) {
34        if (visited_3014.contains(current_3014))
35            return;
36        visited_3014.add(current_3014);
37        System.out.print(current_3014 + " ");
38        for (String neighbor_3014 : graph_3014.getOrDefault(current_3014, new ArrayList<>())) {
39            dfsHelper_3014(neighbor_3014, visited_3014);
40        }
41    }
42    // BFS iteratif
43    public void bfs_3014(String start_3014) {
44        Set<String> visited_3014 = new HashSet<>();
45        Queue<String> queue_3014 = new LinkedList<>();
46        queue_3014.add(start_3014);
47        visited_3014.add(start_3014);
48        System.out.println("Penelusuran BFS:");
49        while (!queue_3014.isEmpty()) {
50            String current_3014 = queue_3014.poll();
51            System.out.print(current_3014 + " ");
52            for (String neighbor_3014 : graph_3014.getOrDefault(current_3014, new ArrayList<>())) {
53                if (!visited_3014.contains(neighbor_3014)) {
54                    queue_3014.add(neighbor_3014);
55                    visited_3014.add(neighbor_3014);
56                }
57            }
58        }
59        System.out.println();
60    }
61    // Main
62    public static void main(String[] args_3014) {
63        GraphTraversal_2511533014 graph_3014 = new GraphTraversal_2511533014();
64
65        // Contoh graf: A-B, A-C, B-D, B-E
66        graph_3014.addEdge_3014("A", "B");
67        graph_3014.addEdge_3014("A", "C");
68        graph_3014.addEdge_3014("B", "D");
69        graph_3014.addEdge_3014("B", "E");
70
71        // Cetak graf awal
72        System.out.println("Graf Awal adalah:");
73        graph_3014.printGraph_3014();
74
75        // Lakukan penelusuran
76        graph_3014.dfs_3014("A");
77        graph_3014.bfs_3014("A");
78    }
79 }
```

Gambar 2.5 Kode Program praktikum “GraphTraversal”

Program GraphTraversal_2511533014 merupakan implementasi struktur data Graph menggunakan Adjacency List. Program ini menyediakan method untuk menambahkan hubungan antar simpul (*edge*), menampilkan graf, serta melakukan penelusuran graf menggunakan algoritma Depth First Search (DFS) dan Breadth First Search (BFS). Pada contoh yang digunakan, graf terdiri dari simpul A, B, C, D, dan E yang saling terhubung.

Berikut penjelasan langkah-langkah kerjanya:

1. `private Map<String, List<String>> graph_3014 = new HashMap<>();`
→ Menyimpan graf dalam bentuk Adjacency List.
2. `public void addEdge_3014(String node1_3014, String node2_3014)`
→ Method untuk menambahkan hubungan (*edge*) antar dua simpul.
3. `graph_3014.putIfAbsent(node1_3014, new ArrayList<>());`
→ Membuat simpul pertama jika belum ada.
4. `graph_3014.putIfAbsent(node2_3014, new ArrayList<>());`
→ Membuat simpul kedua jika belum ada.
5. `graph_3014.get(node1_3014).add(node2_3014);`
→ Menambahkan node2 ke daftar tetangga node1.
6. `graph_3014.get(node2_3014).add(node1_3014);`
→ Menambahkan node1 ke daftar tetangga node2 karena graf bersifat tak berarah.
7. `public void printGraph_3014()`
→ Method untuk menampilkan isi graf.
8. `for (String node_3014 : graph_3014.keySet())`
→ Melakukan perulangan pada seluruh simpul dalam graf.
9. `List<String> neighbors_3014 = graph_3014.get(node_3014);`
→ Mengambil daftar tetangga dari suatu simpul.

10. `System.out.println(String.join(" ", neighbors_3014));`
→ Menampilkan semua tetangga dari simpul tersebut.
11. `public void dfs_3014(String start_3014)`
→ Method untuk melakukan penelusuran DFS dari simpul awal.
12. `Set<String> visited_3014 = new HashSet<>();`
→ Menyimpan simpul yang sudah dikunjungi agar tidak dikunjungi kembali.
13. `dfsHelper_3014(start_3014, visited_3014);`
→ Memanggil method rekursif DFS.
14. `private void dfsHelper_3014(String current_3014, Set<String> visited_3014)`
→ Method rekursif yang menjalankan proses DFS.
15. `if (visited_3014.contains(current_3014))`
→ Memeriksa apakah simpul sudah pernah dikunjungi.
16. `visited_3014.add(current_3014);`
→ Menandai simpul sebagai sudah dikunjungi.
17. `System.out.print(current_3014 + " ");`
→ Menampilkan simpul yang sedang dikunjungi.
18. `for (String neighbor_3014 : graph_3014.getDefault(...))`
→ Menelusuri seluruh tetangga dari simpul saat ini.
19. `dfsHelper_3014(neighbor_3014, visited_3014);`
→ Melanjutkan penelusuran DFS secara rekursif.
20. `public void bfs_3014(String start_3014)`
→ Method untuk melakukan penelusuran BFS dari simpul awal.
21. `Queue<String> queue_3014 = new LinkedList<>();`

- Membuat antrian (queue) yang digunakan oleh BFS.
22. `queue_3014.add(start_3014);`
- Menambahkan simpul awal ke dalam queue.
23. `visited_3014.add(start_3014);`
- Menandai simpul awal sebagai sudah dikunjungi.
24. `while (!queue_3014.isEmpty())`
- Perulangan selama masih ada simpul dalam queue.
25. `String current_3014 = queue_3014.poll();`
- Mengambil simpul pertama dari queue.
26. `System.out.print(current_3014 + " ");`
- Menampilkan simpul yang sedang dikunjungi.
27. `if (!visited_3014.contains(neighbor_3014))`
- Memeriksa apakah tetangga belum pernah dikunjungi.
28. `queue_3014.add(neighbor_3014);`
- Menambahkan tetangga ke queue untuk dikunjungi berikutnya.
29. `visited_3014.add(neighbor_3014);`
- Menandai tetangga sebagai sudah dikunjungi.
30. `public static void main(String[] args_3014)`
- Method utama yang menjalankan program.
31. `GraphTraversal_2511533014 graph_3014 = new GraphTraversal_2511533014();`
- Membuat objek graf baru.
32. `graph_3014.addEdge_3014("A", "B");`
- Menambahkan hubungan antara simpul A dan B.
33. `graph_3014.addEdge_3014("A", "C");`

- Menambahkan hubungan antara simpul A dan C.
34. `graph_3014.addEdge_3014("B", "D");`
- Menambahkan hubungan antara simpul B dan D.
35. `graph_3014.addEdge_3014("B", "E");`
- Menambahkan hubungan antara simpul B dan E.
36. `graph_3014.printGraph_3014();`
- Menampilkan graf dalam bentuk Adjacency List.
37. `graph_3014.dfs_3014("A");`
- Menjalankan penelusuran DFS dimulai dari simpul A.
38. `graph_3014.bfs_3014("A");`
- Menjalankan penelusuran BFS dimulai dari simpul A.

Dari Langkah-langkah diatas kita akan mendapatkan output seperti gambar 2.6 dibawah ini.

```
<terminated> GraphTraversal_2511533014
Graf Awal adalah:
Graf Awal (Adjacency List):
A -> B, C
B -> A, D, E
C -> A
D -> B
E -> B

Penelusuran DFS:
A B D E C
Penelusuran BFS:
A B C D E
```

**Gambar 2.6 Output Kode Program praktikum
Class “GraphTraversal”**

2.1.2 Analisis

Berdasarkan output yang dihasilkan, program berhasil membentuk sebuah graf tak berarah (*undirected graph*) menggunakan representasi *Adjacency List*. Graf terdiri dari lima simpul yaitu **A, B, C, D, dan E** dengan

hubungan A-B, A-C, B-D, dan B-E. Tampilan graf awal menunjukkan bahwa setiap simpul menyimpan daftar tetangga yang terhubung langsung dengannya. Pada proses *Depth First Search (DFS)* yang dimulai dari simpul A, urutan penelusuran yang diperoleh adalah **A B D E C**. Hal ini menunjukkan bahwa algoritma DFS menelusuri satu cabang sedalam mungkin terlebih dahulu sebelum kembali (*backtracking*) untuk mengunjungi cabang lainnya. Sementara itu, pada proses Breadth First Search (BFS) diperoleh urutan **A B C D E**, yang menunjukkan bahwa BFS mengunjungi simpul berdasarkan tingkat kedekatannya dari simpul awal menggunakan struktur data queue. Hasil traversal DFS dan BFS sesuai dengan teori serta struktur graf yang telah dibangun, sehingga dapat disimpulkan bahwa implementasi algoritma penelusuran graf pada program telah berjalan dengan benar.

2.6 Laporan Tugas Praktikum

2.6.1 Tugas Praktikum Class “PetaFTI_2511533014”

- Source Code

```

3 package pekan9_2511533014;
4 import java.awt.EventQueue;
5 import javax.swing.JFrame;
6 import javax.swing.JPanel;
7 import javax.swing.border.EmptyBorder;
8 import javax.swing.*;
9 import java.awt.*;
10 import java.util.*;
11 import java.util.List;
12
13 public class PetaFTI_2511533014 extends JFrame {
14     private JComboBox<String> startBox_3014, goalBox_3014;
15     private JTextArea hasilArea_3014;
16     private GraphPanel_3014 graphPanel_3014;
17     private Map<String, List<String>> graph_3014 = new HashMap<>();
18     public PetaFTI_2511533014() {
19         getContentPane().setBackground(new Color(192, 192, 192));
20         setTitle("Pencarian Jalur Menggunakan BFS dan DFS");
21         setSize(900, 650);
22         setDefaultCloseOperation(EXIT_ON_CLOSE);
23         setLocationRelativeTo(null);
24
25         initGraph_3014();
26
27         JPanel top_3014 = new JPanel();
28         startBox_3014 = new
JComboBox<>(graph_3014.keySet().toArray(new String[0]));

```



```

29         goalBox_3014 = new
JComboBox<>(graph_3014.keySet().toArray(new String[0]));
30
31         JButton bfsBtn_3014 = new JButton("BFS");
32         bfsBtn_3014.setBackground(new Color(0, 128, 6));
33         JButton dfsBtn_3014 = new JButton("DFS");
34         dfsBtn_3014.setBackground(new Color(244, 146, 11));
35         JButton resetBtn_3014 = new JButton("RESET");
36         resetBtn_3014.setBackground(new Color(242, 36, 13));
37
38         top_3014.add(new JLabel("Lokasi Awal"));
39         top_3014.add(startBox_3014);
40         top_3014.add(new JLabel("Lokasi Tujuan"));
41         top_3014.add(goalBox_3014);
42         top_3014.add(bfsBtn_3014);
43         top_3014.add(dfsBtn_3014);
44         top_3014.add(resetBtn_3014);
45
46         graphPanel_3014 = new GraphPanel_3014();
47
48         hasilArea_3014 = new JTextArea(8, 40);
49         hasilArea_3014.setEditable(false);
50
51         getContentPane().add(top_3014, BorderLayout.NORTH);
52         getContentPane().add(graphPanel_3014, BorderLayout.CENTER);
53         getContentPane().add(new JScrollPane(hasilArea_3014),
BorderLayout.SOUTH);
54
55         bfsBtn_3014.addActionListener(e -> runBFS_3014());
56         dfsBtn_3014.addActionListener(e -> runDFS_3014());
57         resetBtn_3014.addActionListener(e -> {
58             graphPanel_3014.reset_3014();
59             hasilArea_3014.setText("");
60         });
61     }
62     private void addEdge_3014(String a, String b) {
63         graph_3014.computeIfAbsent(a, k -> new
ArrayList<>()).add(b);
64         graph_3014.computeIfAbsent(b, k -> new
ArrayList<>()).add(a);
65     }
66     private void initGraph_3014() {
67         addEdge_3014("Gerbang FTI", "Kantin");
68         addEdge_3014("Kantin", "Musholla");
69         addEdge_3014("Musholla", "Lab Algoritma");
70         addEdge_3014("Lab Algoritma", "Lab AI&DS");
71         addEdge_3014("Lab AI&DS", "Aula");
72         addEdge_3014("Aula", "Perpustakaan");
73         addEdge_3014("Perpustakaan", "Akademik");
74         addEdge_3014("Akademik", "Dekanat");
75         addEdge_3014("Dekanat", "Ruang Dosen");
76         addEdge_3014("Ruang Dosen", "Lab Algoritma");
77         addEdge_3014("Dekanat", "Aula");
78         addEdge_3014("Perpustakaan", "Lab Algoritma");
79         addEdge_3014("Kantin", "Aula");
80         addEdge_3014("Musholla", "Perpustakaan");
81         addEdge_3014("Ruang Dosen", "Lab AI&DS");

```

```

82     }
83     private void runBFS_3014() {
84         search_3014(true);
85     }
86     private void runDFS_3014() {
87         search_3014(false);
88     }
89     private void search_3014(boolean bfs_3014) {
90         String start_3014 = (String)
startBox_3014.getSelectedItem();
91         String goal_3014 = (String) goalBox_3014.getSelectedItem();
92
93         Set<String> visited_3014 = new LinkedHashSet<>();
94         Map<String, String> parent_3014 = new HashMap<>();
95
96         if (bfs_3014) {
97             Queue<String> q = new LinkedList<>();
98             q.add(start_3014);
99             visited_3014.add(start_3014);
100
101             while (!q.isEmpty()) {
102                 String cur = q.poll();
103                 if (cur.equals(goal_3014)) break;
104
105                 for (String n : graph_3014.get(cur)) {
106                     if (!visited_3014.contains(n)) {
107                         visited_3014.add(n);
108                         parent_3014.put(n, cur);
109                         q.add(n);
110                     }
111                 }
112             }
113         } else {
114             Stack<String> st = new Stack<>();
115             st.push(start_3014);
116             visited_3014.add(start_3014);
117
118             while (!st.isEmpty()) {
119                 String cur = st.pop();
120                 if (cur.equals(goal_3014)) break;
121
122                 for (String n : graph_3014.get(cur)) {
123                     if (!visited_3014.contains(n)) {
124                         visited_3014.add(n);
125                         parent_3014.put(n, cur);
126                         st.push(n);
127                     }
128                 }
129             }
130         }
131
132         List<String> path_3014 = new ArrayList<>();
133         String cur_3014 = goal_3014;
134         while (cur_3014 != null) {
135             path_3014.add(cur_3014);
136             cur_3014 = parent_3014.get(cur_3014);
137         }

```

```

138     Collections.reverse(path_3014);
139
140     hasilArea_3014.setText(
141         "Metode : " + (bfs_3014 ? "BFS" : "DFS") + "\n\n" +
142         "Jalur : " + String.join(" -> ", path_3014) + "\n\n"
143         +
144         "Node Dikunjungi : " + visited_3014 + "\n\n" +
145         "Jumlah Node Dieksplorasi : " + visited_3014.size()
146     );
147     graphPanel_3014.updateVisited_3014(visited_3014, goal_3014);
148 }
149
150 class GraphPanel_3014 extends JPanel {
151     Map<String, Point> pos_3014 = new HashMap<>();
152     Set<String> visited_3014 = new HashSet<>();
153     String goal_3014 = "";
154
155     GraphPanel_3014() {
156         pos_3014.put("Gerbang FTI", new Point(120, 300));
157         pos_3014.put("Kantin", new Point(220, 260));
158         pos_3014.put("Musholla", new Point(320, 220));
159         pos_3014.put("Lab Algoritma", new Point(430, 180));
160         pos_3014.put("Lab AI&DS", new Point(580, 220));
161         pos_3014.put("Aula", new Point(650, 120));
162         pos_3014.put("Perpustakaan", new Point(500, 80));
163         pos_3014.put("Akademik", new Point(320, 80));
164         pos_3014.put("Dekanat", new Point(180, 120));
165         pos_3014.put("Ruang Dosen", new Point(300, 150));
166     }
167
168     void updateVisited_3014(Set<String> v, String g) {
169         visited_3014 = v;
170         goal_3014 = g;
171         repaint();
172     }
173
174     void reset_3014() {
175         visited_3014.clear();
176         goal_3014 = "";
177         repaint();
178     }
179
180     protected void paintComponent(Graphics g) {
181         super.paintComponent(g);
182
183         g.setColor(Color.GRAY);
184         for (String a : graph_3014.keySet()) {
185             for (String b : graph_3014.get(a)) {
186                 Point p1 = pos_3014.get(a);
187                 Point p2 = pos_3014.get(b);
188                 g.drawLine(p1.x, p1.y, p2.x, p2.y);
189             }
190         }
191
192         for (String node : pos_3014.keySet()) {
193             Point p = pos_3014.get(node);

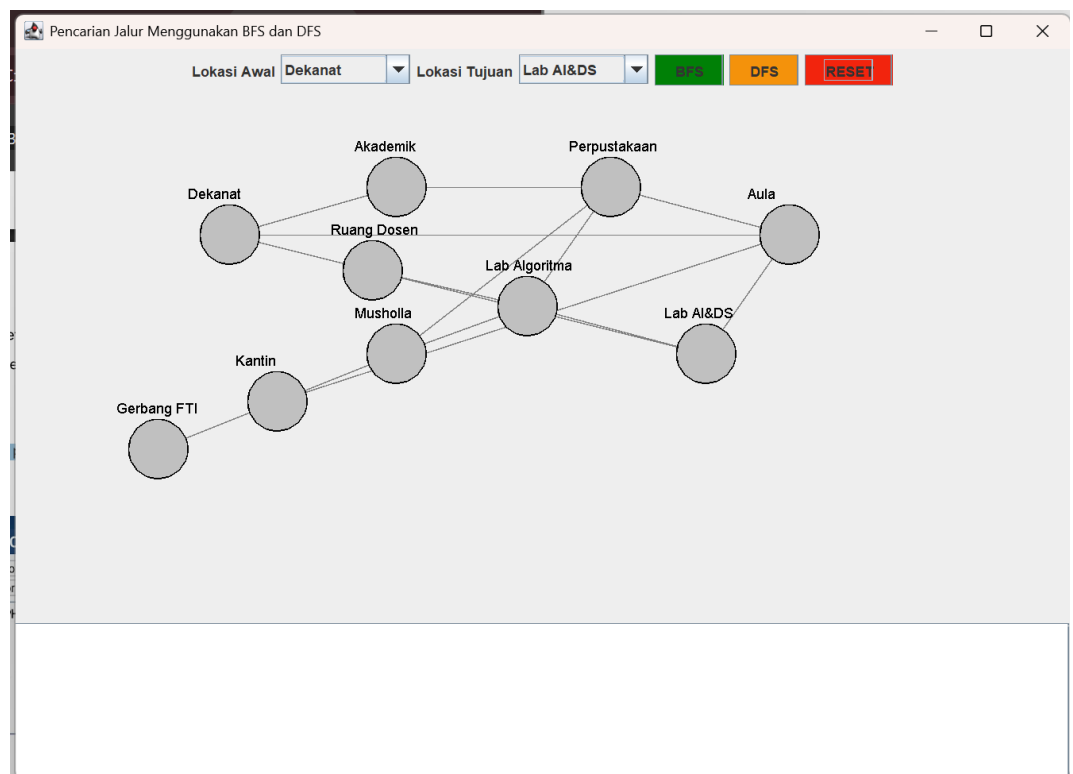
```

```

194
195         if (node.equals(goal_3014))
196             g.setColor(Color.RED);
197         else if (visited_3014.contains(node))
198             g.setColor(Color.GREEN);
199         else
200             g.setColor(Color.LIGHT_GRAY);
201
202         g.fillOval(p.x - 25, p.y - 25, 50, 50);
203         g.setColor(Color.BLACK);
204         g.drawOval(p.x - 25, p.y - 25, 50, 50);
205         g.drawString(node, p.x - 35, p.y - 30);
206     }
207 }
208 }
209
210 public static void main(String[] args) {
211     SwingUtilities.invokeLater(() -> new
212     PetaFTI_2511533014().setVisible(true));
213 }
214

```

- **Output Program**



- **Deskripsi**

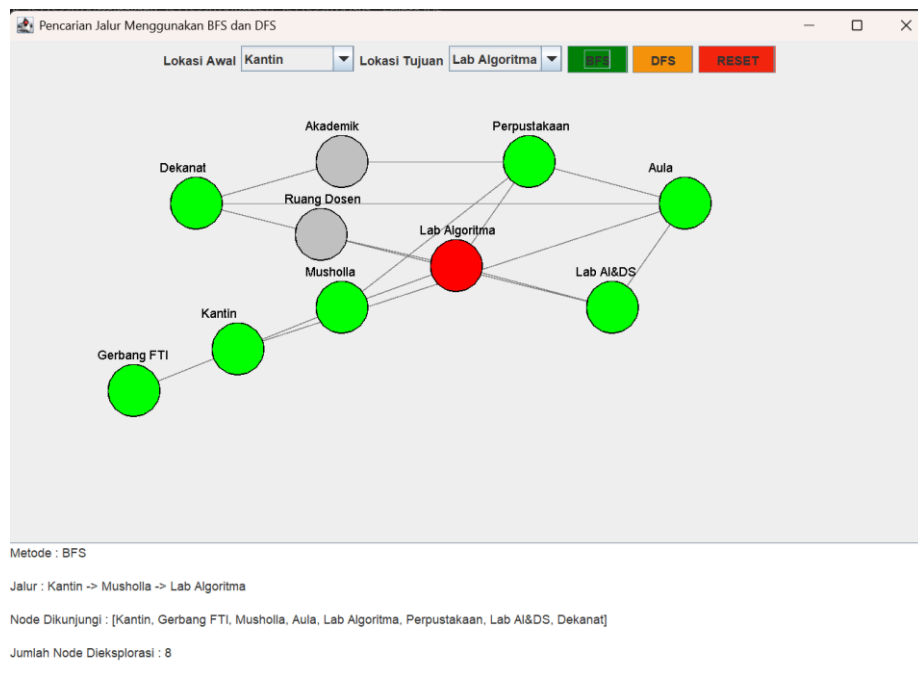
Program PetaFTI_2511533014 merupakan aplikasi GUI berbasis Java Swing yang mengimplementasikan algoritma Breadth First Search (BFS) dan Depth First Search (DFS) untuk mencari jalur pada peta Fakultas Teknologi Informasi Universitas Andalas yang direpresentasikan dalam bentuk graph. Pengguna dapat memilih lokasi awal (*start node*) dan lokasi tujuan (*goal node*) melalui ComboBox, kemudian menjalankan pencarian menggunakan BFS atau DFS. Program akan menampilkan jalur yang ditemukan, urutan node yang dikunjungi, jumlah node yang dieksplorasi, serta memberikan visualisasi graph dengan pewarnaan berbeda pada node yang telah dikunjungi dan node tujuan.

- **Jumlah node dan edge**

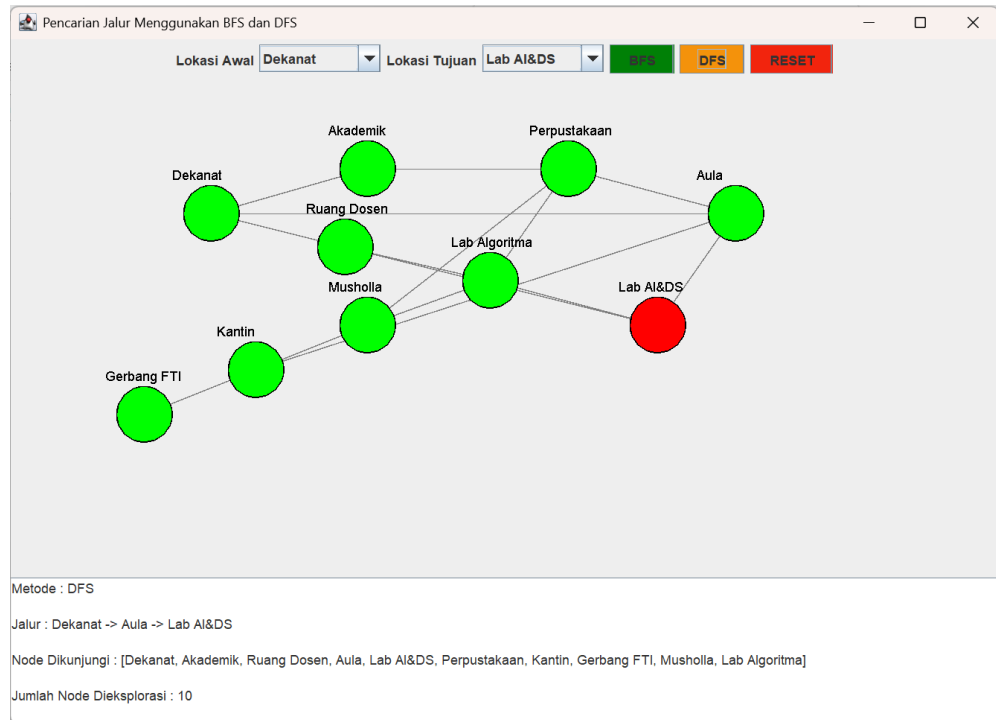
- Node (Vertex) = 10
 1. Gerbang FTI
 2. Kantin
 3. Musholla
 4. Lab Algoritma
 5. Lab AI&DS
 6. Aula
 7. Perpustakaan
 8. Akademik
 9. Dekanat
 10. Ruang Dosen
- Edge (Sisi) = 15
 1. Gerbang FTI ↔ Kantin
 2. Kantin ↔ Musholla
 3. Musholla ↔ Lab Algoritma
 4. Lab Algoritma ↔ Lab AI&DS
 5. Lab AI&DS ↔ Aula
 6. Aula ↔ Perpustakaan
 7. Perpustakaan ↔ Akademik
 8. Akademik ↔ Dekanat

9. Dekanat ↔ Ruang Dosen
10. Ruang Dosen ↔ Lab Algoritma
11. Dekanat ↔ Aula
12. Perpustakaan ↔ Lab Algoritma
13. Kantin ↔ Aula
14. Musholla ↔ Perpustakaan
15. Ruang Dosen ↔ Lab AI&DS

- **Hasil BFS**



- **Hasil DFS**



- **Kesimpulan perbandingan BFS dan DFS**

Berdasarkan implementasi pada program, algoritma *Breadth First Search (BFS)* melakukan pencarian dengan menelusuri node berdasarkan tingkat kedekatannya dari node awal menggunakan struktur data *Queue*. BFS memiliki keunggulan dalam menemukan jalur terpendek pada graph tak berbobot, tetapi biasanya mengeksplorasi lebih banyak node dan membutuhkan memori yang lebih besar. Sementara itu, *Depth First Search (DFS)* melakukan pencarian dengan menelusuri satu cabang sedalam mungkin terlebih dahulu menggunakan *Stack* sebelum kembali ke cabang lain. DFS cenderung menggunakan memori yang lebih sedikit dan dapat menemukan tujuan lebih cepat pada kondisi tertentu, namun tidak selalu menghasilkan jalur terpendek. Oleh karena itu, BFS lebih cocok digunakan ketika tujuan utama adalah mencari jalur terpendek, sedangkan DFS lebih cocok digunakan untuk penelusuran graph secara mendalam dengan kebutuhan memori yang lebih efisien.

BAB III PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa Binary Tree dan Graph merupakan struktur data non-linear yang mampu merepresentasikan hubungan data secara lebih kompleks dibandingkan struktur data linear. Pada Binary Tree, data disusun secara hierarkis dan dapat ditelusuri menggunakan metode Preorder, Inorder, dan Postorder. Sedangkan pada Graph, hubungan antar simpul direpresentasikan melalui vertex dan edge yang dapat ditelusuri menggunakan algoritma BFS dan DFS.

Implementasi Binary Tree dan Graph Traversal menggunakan Java menunjukkan bahwa kedua struktur data tersebut sangat efektif dalam pengelolaan dan pencarian data. BFS melakukan penelusuran berdasarkan tingkat kedekatan node menggunakan queue, sedangkan DFS menelusuri node secara mendalam menggunakan stack atau rekursi. Dengan memahami kedua struktur data ini, mahasiswa dapat mengembangkan kemampuan dalam menyelesaikan berbagai permasalahan yang berkaitan dengan pencarian dan representasi hubungan data.

DAFTAR PUSTAKA

- [1] GeeksforGeeks, "Binary Tree Data Structure," GeeksforGeeks, 2025. [Online]. Available: <https://www.geeksforgeeks.org/binary-tree-data-structure/>. [Accessed: 12-Jun-2026].
- [2] GeeksforGeeks, "Tree Traversals (Inorder, Preorder and Postorder)," GeeksforGeeks, 2025. [Online]. Available: <https://www.geeksforgeeks.org/tree-traversals-inorder-preorder-and-postorder/>. [Accessed: 12-Jun-2026].
- [3] GeeksforGeeks, "Breadth First Search or BFS for a Graph," GeeksforGeeks, 2025. [Online]. Available: <https://www.geeksforgeeks.org/breadth-first-search-or-bfs-for-a-graph/>. [Accessed: 12-Jun-2026].
- [4] GeeksforGeeks, "Depth First Search or DFS for a Graph," GeeksforGeeks, 2025. [Online]. Available: <https://www.geeksforgeeks.org/depth-first-search-or-dfs-for-a-graph/>. [Accessed: 12-Jun-2026].